In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

1.Data Overview

In [2]:
```
# Read csv file
df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/gapminderDataFiveYear.csv')

# Display the first few rows of the dataframe
df.head()
```

Out[2]:

	country	year	pop	continent	lifeExp	gdpPercap
0	Afghanistan	1952	8425333.0	Asia	28.801	779.445314
1	Afghanistan	1957	9240934.0	Asia	30.332	820.853030
2	Afghanistan	1962	10267083.0	Asia	31.997	853.100710
3	Afghanistan	1967	11537966.0	Asia	34.020	836.197138
4	Afghanistan	1972	13079460.0	Asia	36.088	739.981106

In [3]:
```
# Last 5 rows of the dataframe
df.tail()
```

Out[3]:

	country	year	pop	continent	lifeExp	gdpPercap
1699	Zimbabwe	1987	9216418.0	Africa	62.351	706.157306
1700	Zimbabwe	1992	10704340.0	Africa	60.377	693.420786
1701	Zimbabwe	1997	11404948.0	Africa	46.809	792.449960
1702	Zimbabwe	2002	11926563.0	Africa	39.989	672.038623
1703	Zimbabwe	2007	12311143.0	Africa	43.487	469.709298

In [4]:
```
# Display the shape of the dataframe
df.shape
```

Out[4]:
```
(1704, 6)
```

In [ ]:
```
# Total cells in the DataFrame
df.size
```

Out[ ]:
```
11928
```

In [ ]:
```
# Total number of rows in the DataFrame
len(df)
```

Out[ ]:
```
1704
```

In [6]:
```
# Display the data types of each column
df.dtypes
```

Out[6]:
```
country       object
year           int64
pop          float64
continent     object
lifeExp      float64
gdpPercap    float64
dtype: object
```

In [7]:
```
# Display the summary statistics of the dataframe
df.describe()
```

```
#
```

Out[7]:

	year	pop	lifeExp	gdpPercap
count	1704.00000	1.704000e+03	1704.000000	1704.000000
mean	1979.50000	2.960121e+07	59.474439	7215.327081
std	17.26533	1.061579e+08	12.917107	9857.454543
min	1952.00000	6.001100e+04	23.599000	241.165876
25%	1965.75000	2.793664e+06	48.198000	1202.060309
50%	1979.50000	7.023596e+06	60.712500	3531.846988
75%	1993.25000	1.958522e+07	70.845500	9325.462346
max	2007.00000	1.318683e+09	82.603000	113523.132900

In [8]:
```
# display the unique values in the 'continent' column
df['continent'].unique()
```

Out[8]:
```
array(['Asia', 'Europe', 'Africa', 'Americas', 'Oceania'], dtype=object)
```

In [9]:
```
#Display the number of unique values in the 'continent' column
df['continent'].unique().size
```

Out[9]:
```
5
```

In [10]:
```
# Display the unique values in the 'country' column
df['country'].unique()
```

Out[10]:
```
array(['Afghanistan', 'Albania', 'Algeria', 'Angola', 'Argentina',
       'Australia', 'Austria', 'Bahrain', 'Bangladesh', 'Belgium',
       'Benin', 'Bolivia', 'Bosnia and Herzegovina', 'Botswana', 'Brazil',
       'Bulgaria', 'Burkina Faso', 'Burundi', 'Cambodia', 'Cameroon',
       'Canada', 'Central African Republic', 'Chad', 'Chile', 'China',
       'Colombia', 'Comoros', 'Congo, Dem. Rep.', 'Congo, Rep.',
       'Costa Rica', "Cote d'Ivoire", 'Croatia', 'Cuba', 'Czech Republic',
       'Denmark', 'Djibouti', 'Dominican Republic', 'Ecuador', 'Egypt',
       'El Salvador', 'Equatorial Guinea', 'Eritrea', 'Ethiopia',
       'Finland', 'France', 'Gabon', 'Gambia', 'Germany', 'Ghana',
       'Greece', 'Guatemala', 'Guinea', 'Guinea-Bissau', 'Haiti',
       'Honduras', 'Hong Kong, China', 'Hungary', 'Iceland', 'India',
       'Indonesia', 'Iran', 'Iraq', 'Ireland', 'Israel', 'Italy',
       'Jamaica', 'Japan', 'Jordan', 'Kenya', 'Korea, Dem. Rep.',
       'Korea, Rep.', 'Kuwait', 'Lebanon', 'Lesotho', 'Liberia', 'Libya',
       'Madagascar', 'Malawi', 'Malaysia', 'Mali', 'Mauritania',
       'Mauritius', 'Mexico', 'Mongolia', 'Montenegro', 'Morocco',
       'Mozambique', 'Myanmar', 'Namibia', 'Nepal', 'Netherlands',
       'New Zealand', 'Nicaragua', 'Niger', 'Nigeria', 'Norway', 'Oman',
       'Pakistan', 'Panama', 'Paraguay', 'Peru', 'Philippines', 'Poland',
       'Portugal', 'Puerto Rico', 'Reunion', 'Romania', 'Rwanda',
       'Sao Tome and Principe', 'Saudi Arabia', 'Senegal', 'Serbia',
       'Sierra Leone', 'Singapore', 'Slovak Republic', 'Slovenia',
       'Somalia', 'South Africa', 'Spain', 'Sri Lanka', 'Sudan',
       'Swaziland', 'Sweden', 'Switzerland', 'Syria', 'Taiwan',
       'Tanzania', 'Thailand', 'Togo', 'Trinidad and Tobago', 'Tunisia',
       'Turkey', 'Uganda', 'United Kingdom', 'United States', 'Uruguay',
       'Venezuela', 'Vietnam', 'West Bank and Gaza', 'Yemen, Rep.',
       'Zambia', 'Zimbabwe'], dtype=object)
```

In [11]:
```
# display the number of unique values in the 'country' column
df['country'].unique().size
```

Out[11]:
```
142
```

In [15]:
```
df.count()
```

Out[15]:
```
country      1704
year         1704
pop          1704
continent    1704
lifeExp      1704
gdpPercap    1704
dtype: int64
```

In [16]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1704 entries, 0 to 1703
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   country    1704 non-null   object
 1   year       1704 non-null   int64
 2   pop        1704 non-null   float64
 3   continent  1704 non-null   object
 4   lifeExp    1704 non-null   float64
 5   gdpPercap  1704 non-null   float64
dtypes: float64(3), int64(1), object(2)
memory usage: 80.0+ KB
```

2. Column & Row Operations

In [17]:
```
df.columns
```

Out[17]:
```
Index(['country', 'year', 'pop', 'continent', 'lifeExp', 'gdpPercap'], dtype='object')
```

In [18]:
```
df.rename(columns={'year': 'Year', 'pop': 'Population'}, inplace=True)
```

In [20]:
```
# Select columns
df['Population']
```

Out[20]:
```
0        8425333.0
1        9240934.0
2       10267083.0
3       11537966.0
4       13079460.0
           ...
1699     9216418.0
1700    10704340.0
1701    11404948.0
1702    11926563.0
1703    12311143.0
Name: Population, Length: 1704, dtype: float64
```

In [21]:
```
# Select multiple columns
df[['country', 'Year']]
```

Out[21]:

	country	Year
0	Afghanistan	1952
1	Afghanistan	1957
2	Afghanistan	1962
3	Afghanistan	1967
4	Afghanistan	1972
...	...	...
1699	Zimbabwe	1987
1700	Zimbabwe	1992
1701	Zimbabwe	1997
1702	Zimbabwe	2002
1703	Zimbabwe	2007

1704 rows × 2 columns

In [22]:
```
# Select by index
df.iloc[0:4, 0:2]
```

Out[22]:

	country	Year
0	Afghanistan	1952
1	Afghanistan	1957
2	Afghanistan	1962
3	Afghanistan	1967

In [23]:
```
# Select by label
df.loc[0:5, ['country', 'Year']]
```

Out[23]:

	country	Year
0	Afghanistan	1952
1	Afghanistan	1957
2	Afghanistan	1962
3	Afghanistan	1967
4	Afghanistan	1972
5	Afghanistan	1977

In [24]:
```
# Fast single Value
df.at[2, 'country']
```

Out[24]: 'Afghanistan'

3.Filtering And Sorting

In [25]:
```
# Boolean filtering
# filter rows where continent is 'Asia'
```

```
df[df['continent']== 'Asia']
```

Out[25]:

	country	Year	Population	continent	lifeExp	gdpPercap
0	Afghanistan	1952	8425333.0	Asia	28.801	779.445314
1	Afghanistan	1957	9240934.0	Asia	30.332	820.853030
2	Afghanistan	1962	10267083.0	Asia	31.997	853.100710
3	Afghanistan	1967	11537966.0	Asia	34.020	836.197138
4	Afghanistan	1972	13079460.0	Asia	36.088	739.981106
...	...	...	...	...	...	...
1675	Yemen, Rep.	1987	11219340.0	Asia	52.922	1971.741538
1676	Yemen, Rep.	1992	13367997.0	Asia	55.599	1879.496673
1677	Yemen, Rep.	1997	15826497.0	Asia	58.020	2117.484526
1678	Yemen, Rep.	2002	18701257.0	Asia	60.308	2234.820827
1679	Yemen, Rep.	2007	22211743.0	Asia	62.698	2280.769906

396 rows × 6 columns

In [26]:
```
df[df['country']== 'India']
```

Out[26]:

	country	Year	Population	continent	lifeExp	gdpPercap
696	India	1952	3.720000e+08	Asia	37.373	546.565749
697	India	1957	4.090000e+08	Asia	40.249	590.061996
698	India	1962	4.540000e+08	Asia	43.605	658.347151
699	India	1967	5.060000e+08	Asia	47.193	700.770611
700	India	1972	5.670000e+08	Asia	50.651	724.032527
701	India	1977	6.340000e+08	Asia	54.208	813.337323
702	India	1982	7.080000e+08	Asia	56.596	855.723538
703	India	1987	7.880000e+08	Asia	58.553	976.512676
704	India	1992	8.720000e+08	Asia	60.223	1164.406809
705	India	1997	9.590000e+08	Asia	61.765	1458.817442
706	India	2002	1.034173e+09	Asia	62.879	1746.769454
707	India	2007	1.110396e+09	Asia	64.698	2452.210407

In [27]:
```
# sort row
df.sort_values(by= 'Year', ascending=False)
```

Out[27]:

	country	Year	Population	continent	lifeExp	gdpPercap
1679	Yemen, Rep.	2007	22211743.0	Asia	62.698	2280.769906
35	Algeria	2007	33333216.0	Africa	72.301	6223.367465
1139	Nigeria	2007	135031164.0	Africa	46.859	2013.977305
1127	Niger	2007	12894865.0	Africa	56.867	619.676892
575	Germany	2007	82400996.0	Europe	79.406	32170.374420
...	...	...	...	...	...	...
828	Korea, Dem. Rep.	1952	8865488.0	Asia	50.056	1088.277758
48	Argentina	1952	17876956.0	Americas	62.485	5911.315053
1656	West Bank and Gaza	1952	1030585.0	Asia	43.160	1515.592329
24	Algeria	1952	9279525.0	Africa	43.077	2449.008185
0	Afghanistan	1952	8425333.0	Asia	28.801	779.445314

1704 rows × 6 columns

4.Missing Data Handling

In [30]:
```
# Check for missing values
df.isnull().sum()
```

Out[30]:
```
country      0
Year         0
Population   0
continent    0
lifeExp      0
gdpPercap    0
dtype: int64
```

In [ ]:
```
# Drop Nan Rows
df.dropna()
```

In [ ]:
```
# Fill Nan
df.fillna({'Population': df['Population'].mean()})
```

5.Aggregations

In [31]:
```
# Unique values in 'continent' column
df['continent'].unique()
```

Out[31]:
```
array(['Asia', 'Europe', 'Africa', 'Americas', 'Oceania'], dtype=object)
```

In [32]:
```
# Frequency of unique values in 'continent' column
df['continent'].value_counts()
```

Out[32]:
```
continent
Africa      624
Asia        396
Europe      360
Americas    300
Oceania      24
Name: count, dtype: int64
```

In [40]:
```
# Group + Aggregate
# Group by 'continent' and calculate the mean of 'Population'
print('mean =\n',df.groupby('continent')['Population'].mean())

# Group by 'continent' and calculate the sum of 'Population'
print('Sum = \n',df.groupby('continent')['Population'].sum())
```

```
mean =
 continent
Africa      9.916003e+06
Americas    2.450479e+07
Asia        7.703872e+07
Europe      1.716976e+07
Oceania     8.874672e+06
Name: Population, dtype: float64
Sum =
 continent
Africa      6.187586e+09
Americas    7.351438e+09
Asia        3.050733e+10
Europe      6.181115e+09
Oceania     2.129921e+08
Name: Population, dtype: float64
```

In [41]:
```
# Multiple Aggregations
df.groupby('continent').agg({'Population': ['mean', 'sum', 'max', 'min']})
```

Out[41]:

	Population			
	mean	sum	max	min
continent				
Africa	9.916003e+06	6.187586e+09	1.350312e+08	60011.0
Americas	2.450479e+07	7.351438e+09	3.011399e+08	662850.0
Asia	7.703872e+07	3.050733e+10	1.318683e+09	120447.0
Europe	1.716976e+07	6.181115e+09	8.240100e+07	147962.0
Oceania	8.874672e+06	2.129921e+08	2.043418e+07	1994794.0

6. Joining & Combining

In [45]:
```
# Stack DataFrames
df1 = df[['country', 'Year', 'Population']]
```

```
df2 = df[['country', 'Year', 'continent']]

# concatenate DataFrames
df_combined = pd.concat([df1, df2], axis=1)
df_combined.head()
```

Out[45]:

	country	Year	Population	country	Year	continent
0	Afghanistan	1952	8425333.0	Afghanistan	1952	Asia
1	Afghanistan	1957	9240934.0	Afghanistan	1957	Asia
2	Afghanistan	1962	10267083.0	Afghanistan	1962	Asia
3	Afghanistan	1967	11537966.0	Afghanistan	1967	Asia
4	Afghanistan	1972	13079460.0	Afghanistan	1972	Asia

In [ ]:

```
# SQL - Style Join
df1 = df[['country', 'Year', 'Population']]
df2 = df[['country', 'Year', 'continent']]

# Merge DataFrames on 'country' and 'Year'
df_merged = pd.merge(df1, df2, on=['country', 'Year'], how='inner')
df_merged.head(10)
```

Out[ ]:

	country	Year	Population	continent
0	Afghanistan	1952	8425333.0	Asia
1	Afghanistan	1957	9240934.0	Asia
2	Afghanistan	1962	10267083.0	Asia
3	Afghanistan	1967	11537966.0	Asia
4	Afghanistan	1972	13079460.0	Asia
5	Afghanistan	1977	14880372.0	Asia
6	Afghanistan	1982	12881816.0	Asia
7	Afghanistan	1987	13867957.0	Asia
8	Afghanistan	1992	16317921.0	Asia
9	Afghanistan	1997	22227415.0	Asia

7. Reshaping

In [50]:
```
# Pivot Table
# Create a pivot table to summarize the data
pivot_table = df.pivot_table(values='Population', index='Year', columns='continent', aggfunc='sum')

# Display the pivot table
print(pivot_table.head())
```

```
continent       Africa     Americas          Asia       Europe     Oceania
Year
1952       237640501.0  345152446.0  1.395357e+09  418120846.0  10686006.0
1957       264837738.0  386953916.0  1.562781e+09  437890351.0  11941976.0
1962       296516865.0  433270254.0  1.696357e+09  460355155.0  13283518.0
1967       335289489.0  480746623.0  1.905663e+09  481178958.0  14600414.0
1972       379879541.0  529384210.0  2.150972e+09  500635059.0  16106100.0
```

In [51]:
```
# Unpivot
# Unpivot the DataFrame
```

```
df_unpivoted = df.melt(id_vars=['country', 'Year'], value_vars=['Population', 'continent'], var_name='Variable', valu
print(df_unpivoted.head())
```

```
       country  Year    Variable       Value
0  Afghanistan  1952  Population   8425333.0
1  Afghanistan  1957  Population   9240934.0
2  Afghanistan  1962  Population  10267083.0
3  Afghanistan  1967  Population  11537966.0
4  Afghanistan  1972  Population  13079460.0
```

8. Exporting Data

In [52]:
```
# Save DataFrame to CSV
df.to_csv('gapminder_data.csv', index=False)
```

In [ ]:
```
# Save Excel
# Excel: A spreadsheet format that is widely used for data storage and analysis
df.to_excel('gapminder_data.xlsx', index=False)

# Save DataFrame to JSON
# JSON: A lightweight data interchange format
df.to_json('gapminder_data.json', orient='records', lines=True)

# Save DataFrame to Parquet
# Parquet: Means columnar storage, which is efficient for large datasets
df.to_parquet('gapminder_data.parquet', index=False)
```

9. Advanced Tricks

In [56]:
```
# SQL-like queries
# Select the rows where 'continent' is 'Asia' and 'Year' is 2007
df.query("continent == 'Asia' and Year == 2007")
```

Out[56]:

	country	Year	Population	continent	lifeExp	gdpPercap
11	Afghanistan	2007	3.188992e+07	Asia	43.828	974.580338
95	Bahrain	2007	7.085730e+05	Asia	75.635	29796.048340
107	Bangladesh	2007	1.504483e+08	Asia	64.062	1391.253792
227	Cambodia	2007	1.413186e+07	Asia	59.723	1713.778686
299	China	2007	1.318683e+09	Asia	72.961	4959.114854
671	Hong Kong, China	2007	6.980412e+06	Asia	82.208	39724.978670
707	India	2007	1.110396e+09	Asia	64.698	2452.210407
719	Indonesia	2007	2.235470e+08	Asia	70.650	3540.651564
731	Iran	2007	6.945357e+07	Asia	70.964	11605.714490
743	Iraq	2007	2.749964e+07	Asia	59.545	4471.061906
767	Israel	2007	6.426679e+06	Asia	80.745	25523.277100
803	Japan	2007	1.274680e+08	Asia	82.603	31656.068060
815	Jordan	2007	6.053193e+06	Asia	72.535	4519.461171
839	Korea, Dem. Rep.	2007	2.330172e+07	Asia	67.297	1593.065480
851	Korea, Rep.	2007	4.904479e+07	Asia	78.623	23348.139730
863	Kuwait	2007	2.505559e+06	Asia	77.588	47306.989780
875	Lebanon	2007	3.921278e+06	Asia	71.993	10461.058680
947	Malaysia	2007	2.482129e+07	Asia	74.241	12451.655800
1007	Mongolia	2007	2.874127e+06	Asia	66.803	3095.772271
1055	Myanmar	2007	4.776198e+07	Asia	62.069	944.000000
1079	Nepal	2007	2.890179e+07	Asia	63.785	1091.359778
1163	Oman	2007	3.204897e+06	Asia	75.640	22316.192870

	country	Year	Population	continent	lifeExp	gdpPercap
1175	Pakistan	2007	1.692706e+08	Asia	65.483	2605.947580
1223	Philippines	2007	9.107729e+07	Asia	71.688	3190.481016
1319	Saudi Arabia	2007	2.760104e+07	Asia	72.777	21654.831940
1367	Singapore	2007	4.553009e+06	Asia	79.972	47143.179640
1439	Sri Lanka	2007	2.037824e+07	Asia	72.396	3970.095407
1499	Syria	2007	1.931475e+07	Asia	74.143	4184.548089
1511	Taiwan	2007	2.317429e+07	Asia	78.400	28718.276840
1535	Thailand	2007	6.506815e+07	Asia	70.616	7458.396327
1655	Vietnam	2007	8.526236e+07	Asia	74.249	2441.576404
1667	West Bank and Gaza	2007	4.018332e+06	Asia	73.422	3025.349798
1679	Yemen, Rep.	2007	2.221174e+07	Asia	62.698	2280.769906

In [58]:
```python
# Apply Functions
# Define a function to categorize population
def categorize_population(population):
    if population < 1000000:
        return 'Low'
    elif population < 10000000:
        return 'Medium'
    else:
        return 'High'

# Apply the function to the 'Population' column
df['Population_Category'] = df['Population'].apply(categorize_population)
# Display the first few rows with the new column
print(df.head())
```

```
       country  Year  Population continent  lifeExp   gdpPercap  \
0  Afghanistan  1952   8425333.0      Asia   28.801  779.445314
1  Afghanistan  1957   9240934.0      Asia   30.332  820.853030
2  Afghanistan  1962  10267083.0      Asia   31.997  853.100710
3  Afghanistan  1967  11537966.0      Asia   34.020  836.197138
4  Afghanistan  1972  13079460.0      Asia   36.088  739.981106

  Population_Category
0              Medium
1              Medium
2                High
3                High
4                High
```

In [59]:
```
# Correlation
# Calculate the correlation between 'Year' and 'Population'
correlation = df['Year'].corr(df['Population'])
print(f'Correlation between Year and Population: {correlation}')
```

```
Correlation between Year and Population: 0.08230807782724023
```

In [61]:
```
# Top N
# Get the top 5 countries by population in 2007
top_n = df[df['Year']== 2007].nlargest(5, 'Population')[['country', 'Population']]
top_n
```

Out[61]:

	country	Population
299	China	1.318683e+09
707	India	1.110396e+09
1619	United States	3.011399e+08
719	Indonesia	2.235470e+08
179	Brazil	1.900106e+08

In [65]:
```
# Memory Check
# Display memory usage of the DataFrame
df.memory_usage(deep=True)
```

```
Out[65]: Index                  132
         country              97728
         Year                 13632
         Population           13632
         continent            93552
         lifeExp              13632
         gdpPercap            13632
         Population_Category  91862
         dtype: int64
```